NETWORK ANOMALY DETECTOR

Raven

Network security for places with no security team.

5

detectors, rules and statistical

462

automated tests

4 / 4

live attacks caught on loopback



REAL SCREENSHOT: LIVE TEST ON LOOPBACK

Runs on one Mac. No cloud. No account.

Small networks are attacked like big ones. Nobody is watching.

No security staff

The admin is also the help desk. Nobody reads logs, and alerts nobody understands get ignored.

Tools built for a SOC

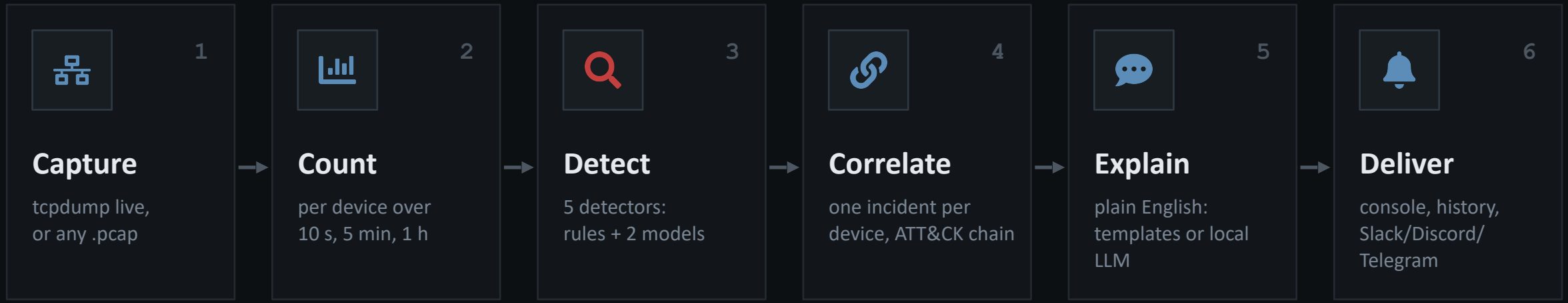
Enterprise products assume analysts, a SIEM and a budget. Small teams cannot run or afford them.

Attacks hide in the noise

A port scan, password guessing, a malware check-in: each is ordinary traffic until something counts it.

Raven: one Mac on the gateway watches the traffic, spots the pattern, and tells the admin in plain English what to look at.

From packets to a plain-English alert in six steps



Sanitiser gate: DNS names and hostnames come from the network, so they are treated as hostile. They are cleaned before any LLM, notification or screen sees them, and the console only ever shows them as plain text.

LLMs never decide: detection is rules and statistics. A local model, if you turn one on, only rewrites the wording, and its output is fact-checked against the evidence.

Five detectors, each tied to an ATT&CK technique

Port scan

T1046

One device probing many ports or hosts, fast or slow.

`400 ports in about 1 s, 99.75% failed`

Brute force

T1110

Many connections to SSH, RDP, SMB or FTP that mostly complete.

`21 logins to port 2222, all completed`

Beaconing

T1071.001

One server contacted at suspiciously regular intervals.

`13 check-ins every 1.0003 s, jitter 0.2%`

DNS tunnelling

T1071.004

Long, random-looking lookups under one domain. An n-gram model scores how random the names look.

`14 TXT lookups, 67 characters, 4.5 bits/char`

Unusual behaviour

statistical

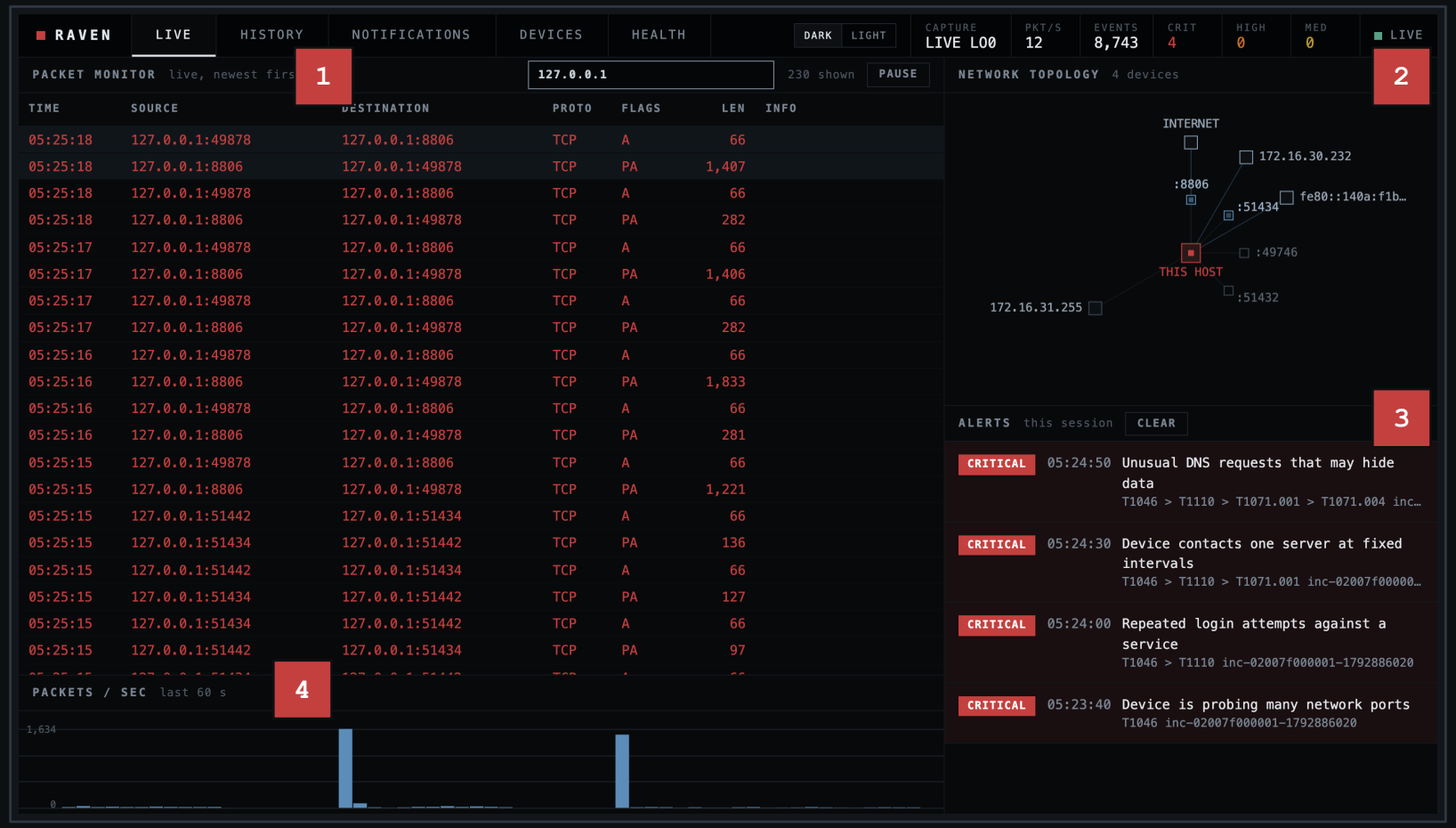
An Isolation Forest learns your network's normal, then flags volume far beyond it.

`bulk upload: bytes_out 263x the baseline`

WHY THE STATISTICAL ONE IS CAPPED

It only knows "unusual", not "attack". Its alerts are capped at MEDIUM, carry no ATT&CK technique, and are hidden while a named attack is already active on that device.

A local console that shows what is happening, now



1 Packet monitor

Every packet event, newest first, with a filter box. A flagged device shows red for a minute.

2 Network topology

This host in the middle, devices around it, service ports as small boxes. A scan lights up the ports.

3 Alert stack

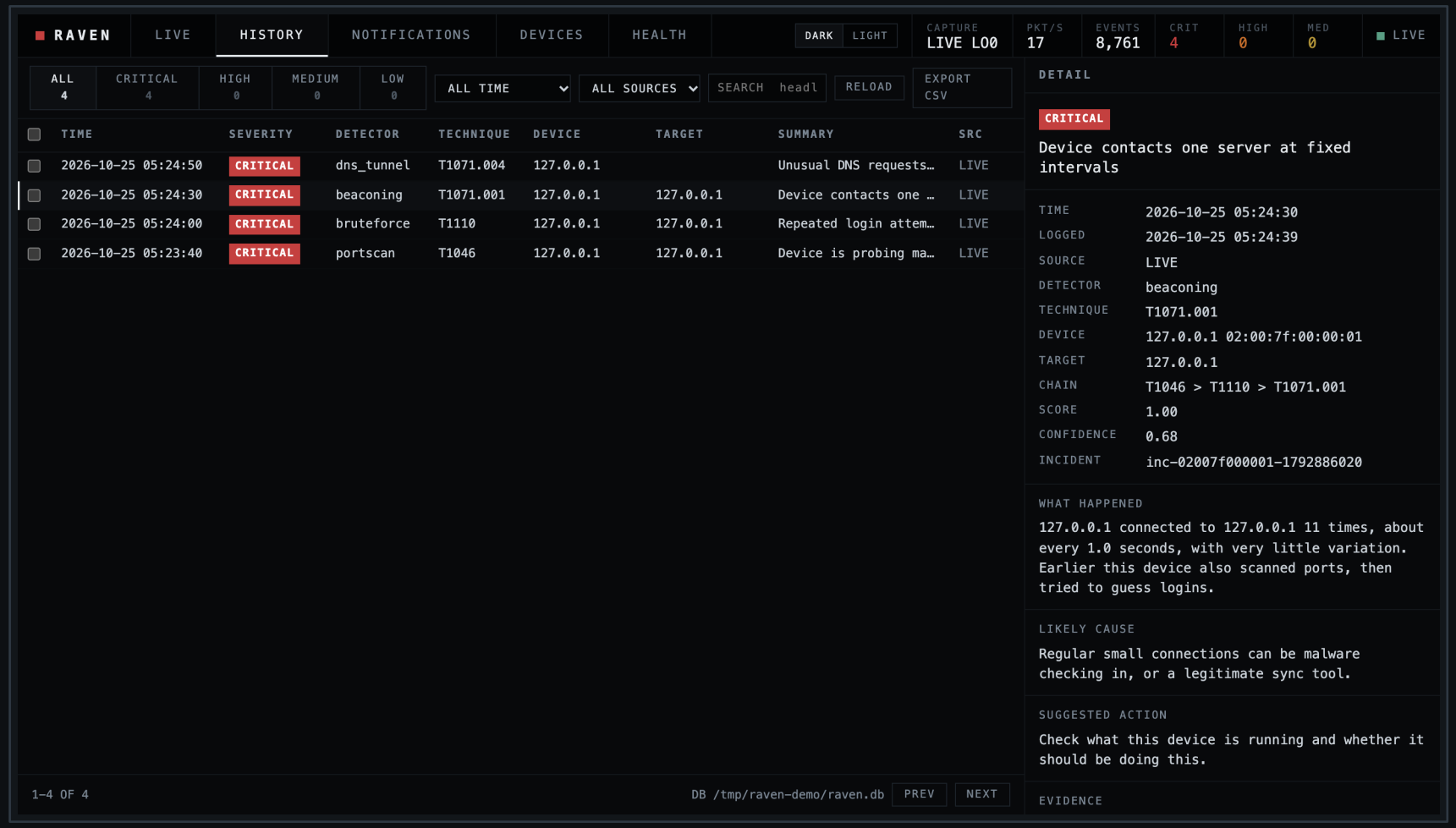
Every CRITICAL stays on screen with its headline and ATT&CK chain.

4 Packets per second

The last 60 seconds, so a burst stands out.

Listens on 127.0.0.1 only. Token needed once. Strict content-security policy.

A permanent history you can search, export and clean up



Survives restarts

SQLite in ~/.raven. The same history from any folder.

Find anything

Filter by severity, time, source. Search headline, IP, technique or incident.

See why

Plain-English explanation plus the numbers that triggered it.

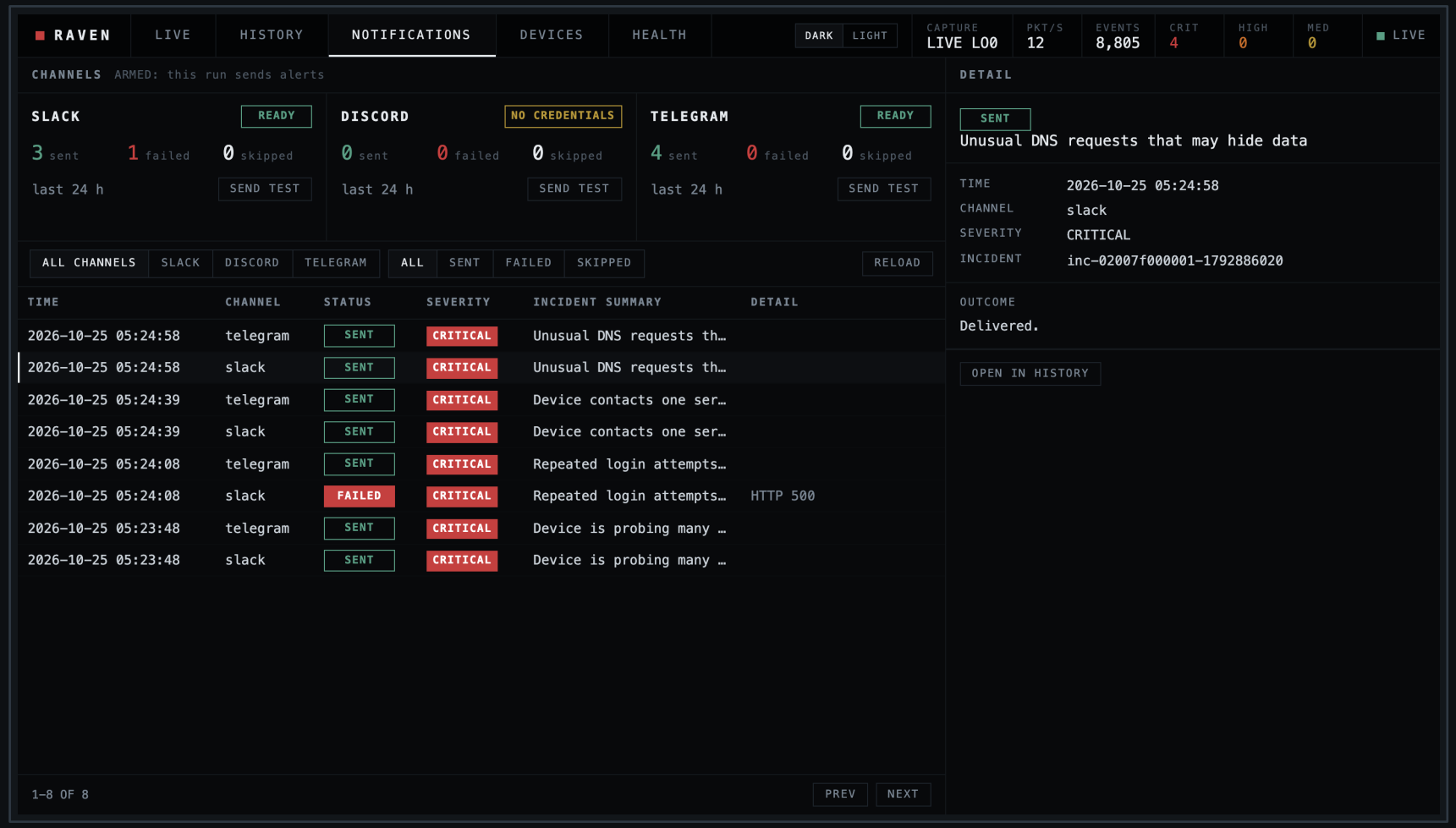
Take it with you

CSV export, or raven history --json for scripts.

Stay in control

Delete chosen entries; every deletion is audited.

Know what was sent, and what failed



Channel status

Ready, no credentials or not in config, with 24 h counts.

Armed or not

A banner says whether this run will actually send alerts.

A log of every send

Headline, severity and, for failures, the reason.

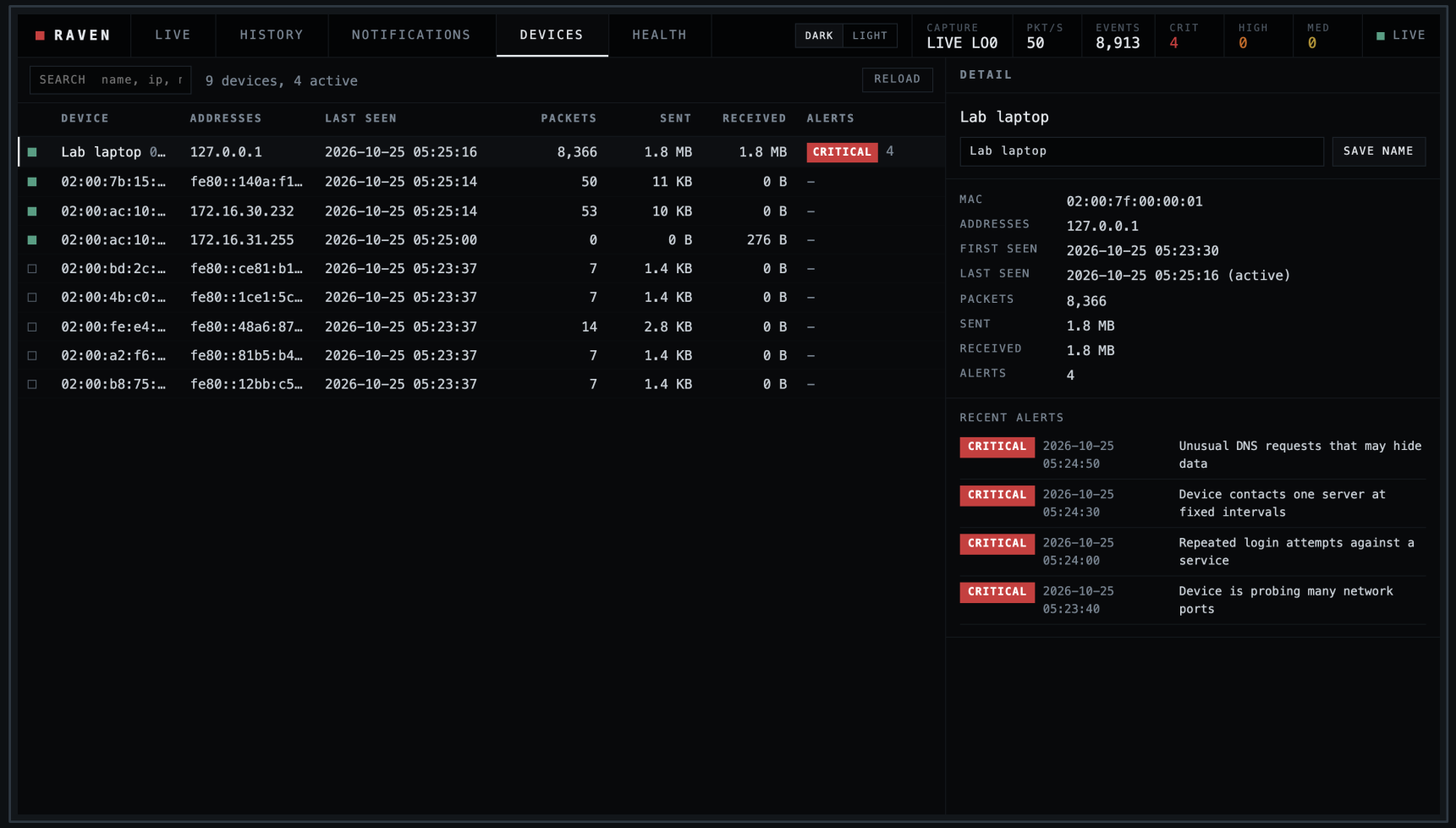
Prove delivery

SEND TEST sends one marked message, after a second click.

Nothing sensitive

Reasons never contain URLs or tokens.

Every device, with its alert record



What the capture has seen

By MAC: addresses, last seen, packets, bytes in and out.

Survives restarts

Stored in the database, not just in memory.

Alerts per device

Worst severity and recent alerts; click to open in History.

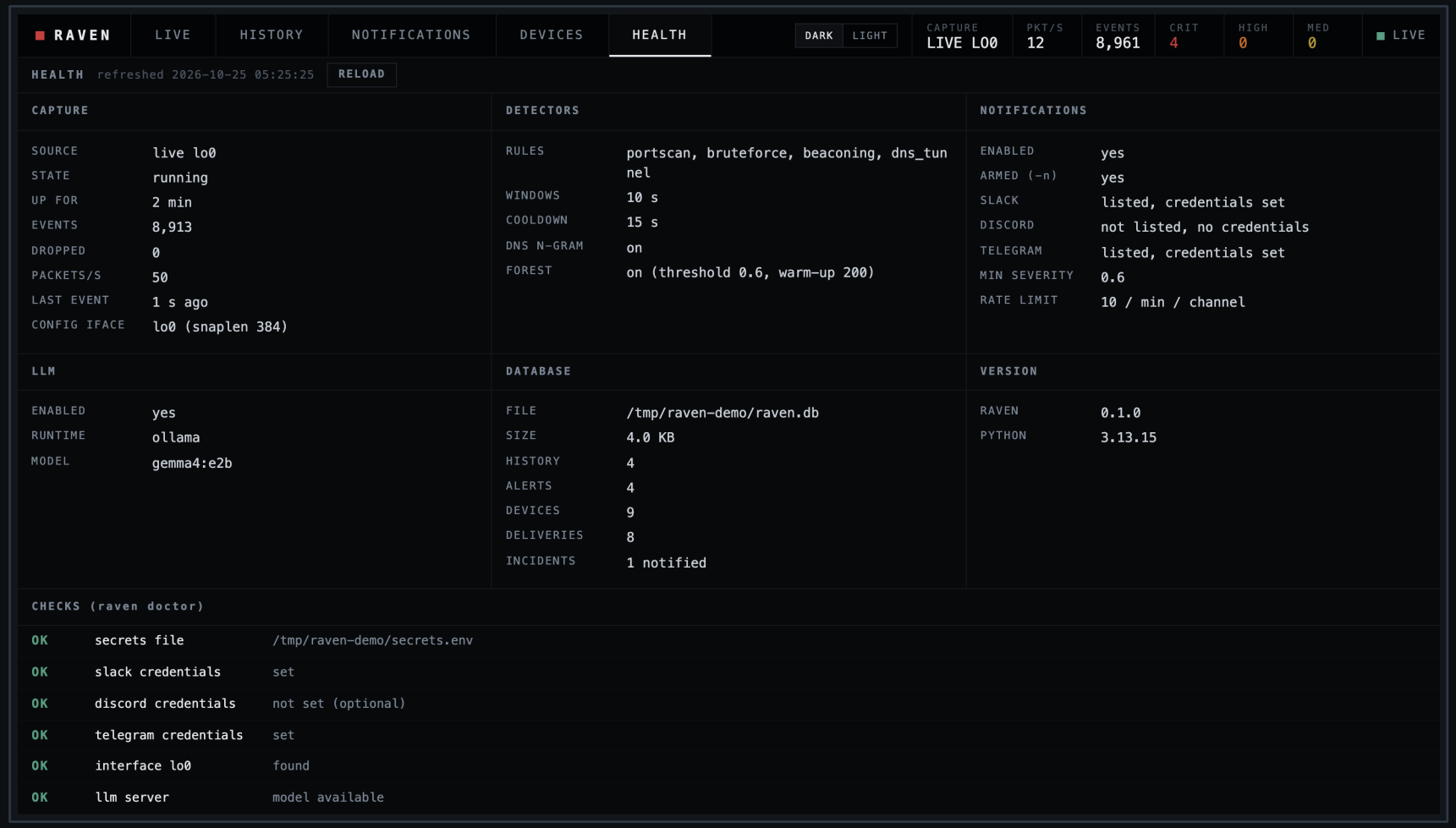
Name it

Your label, saved and audited, never from network data.

No noise

Broadcast and multicast addresses are left out.

Is it healthy? One page answers



Is capture alive

Source, state, uptime and the age of the last packet.

What is switched on

Rules, windows, thresholds and both statistical models.

Setup at a glance

Alerts armed? Which LLM model? Credentials: set or not.

The database

Size and counts of history, devices and deliveries.

Checks built in

The same checks as raven doctor, refreshed live.

Fast attacks and slow ones: three clocks, not one

SLOWEST PORT SCAN EACH CLOCK STILL CATCHES (default thresholds, probes per minute)

10 s window

20 ports in 10 s



fast scan: CRITICAL in the live test

5 min window

15 ports in 5 min



slow scan, 1 probe/s: HIGH in the live test

1 hour window

60 ports in 1 hour



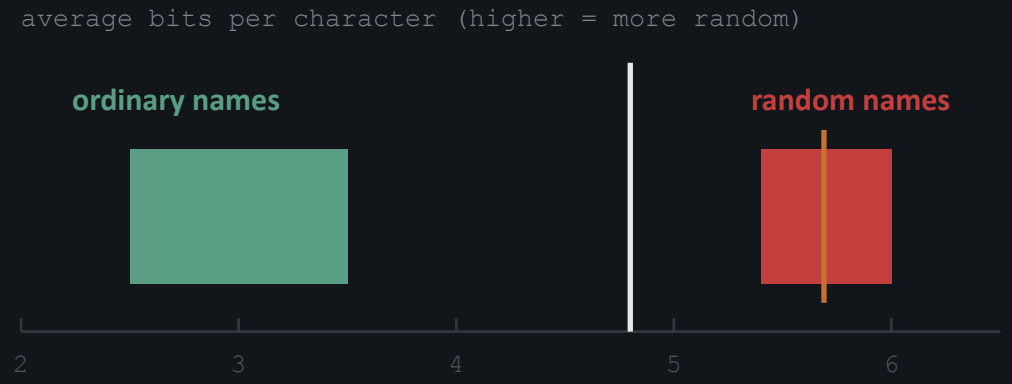
low-and-slow: one probe a minute

bar length uses a square-root scale

One short window sees only the fast attacker. Three clocks also catch the patient one.

Two statistical models catch what the rules miss

DNS N-GRAM / HOW RANDOM DOES A NAME LOOK?



RULES ALONE: MISSED

WITH N-GRAM: CAUGHT

ISOLATION FOREST / IS THIS NORMAL?

40 min → **learns** → **20 s upload**

of normal browsing by 3 devices

what normal looks like on this network

864 KB per 10 s, one long connection



RULES ALONE: MISSED

WITH FOREST: CAUGHT

Measured on synthetic traffic only. Both are on by default and switchable. Normal traffic raised nothing in the 12-network benign test; the false-alarm rate on a real network is not yet known.

Tested live: all four attacks caught, the control stayed quiet

SECONDS FROM ATTACK START TO ALERT (live, loopback, 10 s window)



0

alerts from the control:
ordinary web browsing

4 / 4

beacon runs caught once the capture was confirmed running

Detectors run on a 10 s grid, so an alert lands within one grid step of the threshold being crossed.

Quiet by design: false alarms are what make people stop reading alerts



Two signals, not one

DNS tunnelling needs two independent signals to agree.



Shared services are not C2

A destination or domain that 3+ devices use is treated as a shared service.



Cooldown

Once a detector fires for a device it stays quiet for 5 minutes. No alert storms.



Statistical guard

The forest only fires when a value is over 2x anything in its baseline.



Your allow-lists

trusted_ips, allowed destinations and ignored domains, in one config file.

BENIGN TEST SUITE (SYNTHETIC)

0 alerts

12 seeded networks, 8 devices each, 40 minutes of browsing, shared-service polling, refused connections and hashed CDN names. All with both statistical models on.

Real-network false-alarm rate: not yet measured. A one-day check is on the verification list.

Alerts an admin can act on, not raw numbers

WHAT THE DETECTOR RECORDED

detector	beaconing
technique	T1071.001
dst_port	8080
count	16
mean_interval	1.0003 s
interval_cv	0.0023
mean_bytes	~1 KB

WHAT THE ADMIN READS

CRITICAL

Device contacts one server at fixed intervals

What happened The device connected to the same server 16 times, about every 1.0 seconds, with very little variation.

Likely cause Regular small connections can be malware checking in, or a legitimate sync tool.

Suggested action Check what this device is running and whether it should be doing this.

Templates first

Always available, always matches the evidence.

Optional local LLM

Ollama or llama.cpp, on your machine. Tried with Gemma (e2b) through Ollama.

Fact-checked

Invented facts are rejected; the template is used instead.

Alerts where the admin already is

Slack

incoming webhook

Discord

webhook

Telegram

bot token + chat id

WHAT ARRIVES

[CRITICAL] Device is probing many network ports

Device 192.168.2.30 tried 400 different ports on 192.168.2.20 within 10 seconds, and 99% of the attempts failed. Check what this device is running, or disconnect it if you do not recognise this.

POLICIES THAT PREVENT ALERT FATIGUE

severity floor	de-duplication	rate limit	retry + back-off	circuit breaker
default 0.6	one per chain step	10 / min / channel	2 retries	3 failures, 5 min off

Opt-in, off by default. Keys stay in a private file. Messages never contain packet data. Delivery is tested against fakes; real-channel delivery is still to be confirmed.

A security tool has to be safe to run



Network text is hostile

DNS names and hostnames pass one sanitiser before any LLM, alert or screen. The console shows them as plain text only.



A locked-down console

127.0.0.1 only, per-run token, strict content-security policy, no inline scripts, and a test that fails if HTML is built from strings.



Observe and alert only

Raven never blocks, changes firewall rules or touches the network. A human decides what to do.



Secrets stay private

Webhook URLs and tokens live in a mode 600 file, are never printed or logged, and errors never leak URLs.



Attack scripts stay home

The demo attacks refuse any target except loopback. Own machine only.

SECURITY TEST GROUP

85 tests that must never be empty

They cover hostile DNS names, hostile history filters, SQL text in searches, spreadsheet-formula export, cross-site delete, rename and test-send requests, and planted symlinks.

Found and fixed: an automated review flagged a symlink risk in the sudo file-ownership code; it is fixed and tested.

Who it is for, and where it sits



Sees all traffic that crosses the gateway.
On a plain laptop on Wi-Fi it sees only that laptop.

FOR

- Small teams
- Schools and colleges
- Labs and home offices
- Students learning detection

NOT FOR

- Enterprise scale
- Automatic blocking
- Reading encrypted traffic

HOW IT IS USED

1. Install on the gateway Mac
2. Watch the console, or just wait for alerts
3. Open History to review what happened

From clone to first detection in five minutes

1 INSTALL

```
$ brew install uv
$ git clone https://github.com/MRX-72/Hackt0b3r.git raven && cd raven
$ uv tool install .
$ raven init && raven doctor
```

2 QUICK START (no network needed)

```
$ uv run python tools/make_demo_pcap.py demo.pcap
$ raven replay demo.pcap
$ raven ui --replay demo.pcap
$ raven history
```

3 ATTACK YOURSELF (two terminals)

```
# terminal 1: the monitor
$ raven --config examples/demo-10s.toml ui --iface lo0
# terminal 2: the attacks, on your own machine only
$ uv run python tools/demo_attacks.py
```

TRY IT TODAY

See it catch an attack on your own machine.

```
$ uv tool install .
$ raven init
$ uv run python \
    tools/demo_attacks.py
```

Five minutes, no network needed, nothing leaves your Mac.

Questions?

README.md · USER_GUIDE.md · docs/ABLATION.md